Fast Sampling from Generative Models

Eric Frankel

March 3, 2026

UNIVERSITY of WASHINGTON

The Sampling Bottleneck

Diffusion/flow models achieve **state-of-the-art** image quality, but sampling is **expensive**: 50–250+ neural function evaluations (NFEs).

Two philosophies for efficient sampling:

(A) Make the solver faster

Keep the ODE framework, optimize the solver coefficients

$\longrightarrow$ **S4S** (4–7 NFEs)

(B) Eliminate the solver

Don't solve an ODE at all; learn a single-step pushforward

$\longrightarrow$ **Drifting** (1 NFE)

Background

Diffusion Models Refresher

Forward process (add noise):

$$\mathsf{x}_t = \alpha_t \, \mathsf{x}_0 + \sigma_t \, \boldsymbol{\epsilon}, \qquad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathsf{0}, \mathsf{I})$$

Probability flow ODE (remove noise):

$$d\mathsf{x}_t = \left[f(t) \, \mathsf{x}_t + \frac{g^2(t)}{2\sigma_t} \, \boldsymbol{\epsilon}_\theta(\mathsf{x}_t, t) \right] dt$$

Log signal-to-noise ratio reparametrization:

$$\lambda_t = \log\!\big(\alpha_t / \sigma_t\big)$$

Exact solution (change of variables via λ):

$$\frac{\mathsf{x}_s}{\alpha_s} = \frac{\mathsf{x}_t}{\alpha_t} - \int_{\lambda_t}^{\lambda_s} e^{-\lambda} \, \hat{\boldsymbol{\epsilon}}_\theta(\mathsf{x}_\lambda, \lambda) \, d\lambda$$

ODE Solvers for Diffusion

Taylor-expand the exact integral $\Rightarrow$ family of numerical solvers:

Class	**Examples**	**Idea**
Single-step (SS)	DPM-Solver	Runge-Kutta–style intermediate evaluations
Linear multi-step (LMS)	iPNDM, PNDM	Reuse past ϵ_θ evaluations
Predictor–corrector (PC)	UniPC	Predict then refine

The NFE budget: each evaluation of $\epsilon_\theta(\mathrm{x}_t, t)$ costs **one NFE**.

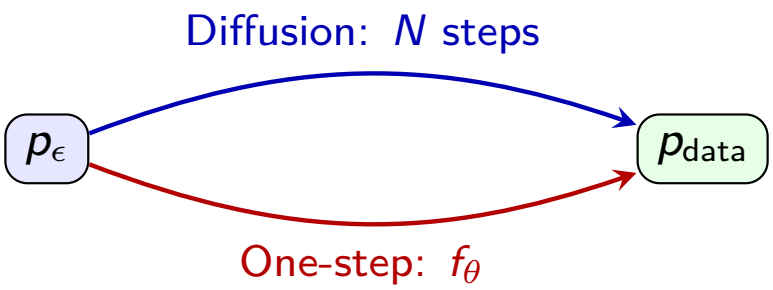
All solvers trade *NFEs* for *accuracy*—fewer steps $\Rightarrow$ larger truncation error.

The Pushforward Perspective

A **generator** defines a pushforward:

$$q = (f_\theta)_\# p_\epsilon, \qquad \text{Goal: } q \approx p_{\text{data}}$$

- **Diffusion models** build this pushforward *iteratively* at inference
  - Compose many small steps of the ODE solver
- **What if** we could do it in *one step*?
  $\mathrm{x} = f_\theta(\boldsymbol{\epsilon}), \quad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathrm{0}, \mathrm{I})$



This frames both approaches we will study.

S4S: Solving for a Diffusion Model Solver

S4S: Motivation

Traditional solvers minimize **local truncation error** (LTE).

In the **few-NFE regime** ($N = 4$–7), step sizes are $\mathcal{O}(1)$, not $\mathcal{O}(1/N)$.

Three **regularity conditions all fail**:

1. **Smoothness:** Higher-order derivatives of ϵ_θ may not exist
2. **Lipschitzness:** Large Lipschitz constants break error propagation
3. **Step size scaling:** $h_{\text{max}} = \mathcal{O}(1)$, not the required $\mathcal{O}(1/N)$

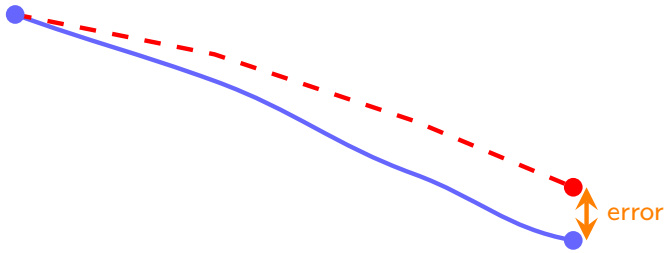
Key insight

Tracking the ODE trajectory is the **wrong objective**—we only care about the *final output*.

From Local to Global Error

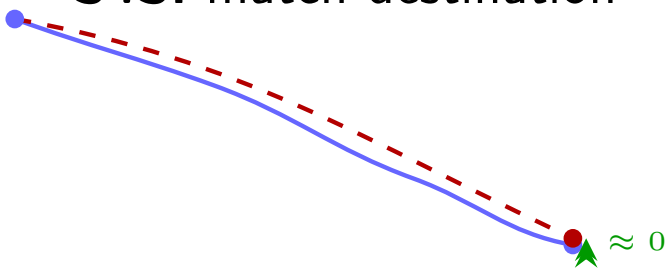
Local error control $\neq$ **global error control** when $h_{\max}$ is large.

Traditional: follow trajectory



Minimize local error at each step

S4S: match destination



Minimize global error (match teacher output)

New objective: directly minimize *global error* by matching a strong teacher solver's output.

General Solver Formulation

Unified update rule (LMS and SS solvers):

$$\tilde{\mathrm{x}}_{t_i} = \frac{\alpha_{t_i}}{\alpha_{t_{i-1}}} \tilde{\mathrm{x}}_{t_{i-1}} - \sigma_{t_i}(e^{h_i} - 1)\,\Delta_i(\phi)$$

LMS increment (reuse past evaluations):

$$\Delta_i = \sum_{j=1}^{k} b_{j,i}\ \epsilon_\theta\big(\tilde{\mathrm{x}}_{t_{i-j}},\ t_{i-j}\big)$$

SS (Runge-Kutta) increment (intermediate evaluations):

$$\Delta_i = \sum_{j=1}^{s} \beta_{j,i}\ \epsilon_\theta\big(\tilde{\mathrm{x}}_{t_i^{(j)}},\ c_{j,i}\big)$$

Learnable: $\phi = \{b_{j,i},\ \beta_{j,i},\ a_{j,i,l},\ c_{j,i}\}$ — few time-dependent coefficients per step.

S4S Objective: Teacher-Student Distillation

Hard objective:

$$\mathcal{L}(\phi) = \mathbb{E}_{\mathsf{x}_T \sim \mathcal{N}(\mathsf{0}, \tilde{\sigma}^2 \mathsf{I})}\Big[\, d\big(\Psi_\phi(\mathsf{x}_T),\ \Psi^*(\mathsf{x}_T)\big)\,\Big]$$

- **Teacher** Ψ^*: strong solver, many NFEs (e.g., UniPC at 20 NFE)
- **Student** Ψ_ϕ: same solver type, few NFEs, learned coefficients
- $d(\cdot,\cdot)$: perceptual distance (LPIPS)

Key advantages

- **Data-free:** only samples from $\mathcal{N}(\mathsf{0}, \tilde{\sigma}^2 \mathsf{I})$
- **Lightweight:** $\sim$6–30 solver coefficients, not the network
- **Fast:** $<$1–4 hours on a single A100

Relaxed Objective

Hard objective is difficult with **very few parameters** (e.g., 6 for LMS at $N=3$).

Relaxed objective—allow input perturbation:

$$\min_{\phi} \mathbb{E}_{\mathsf{x}_T}\left[\min_{\mathsf{x}'_T \in B_r(\mathsf{x}_T)} d\big(\Psi_\phi(\mathsf{x}'_T),\ \Psi^*(\mathsf{x}_T)\big)\right]$$

where $B_r(\mathsf{x}_T) = \{\mathsf{x}'_T : \|\mathsf{x}'_T - \mathsf{x}_T\|_2 \leq r\tilde{\sigma}\}$.

Intuition: a *slightly different* noise input $\rightarrow$ *same-quality* output.

Theory: relaxed objective $= 0 \Rightarrow$ KL bound between student/teacher outputs.

Solved via **alternating projected SGD** over ϕ and x'_T.

S4S-Alt: Joint Optimization of Coefficients & Time Steps

Alternating minimization over coefficients ϕ and time steps Ξ:

For $k = 1, \ldots, K$:

1. **Optimize schedule Ξ** (holding ϕ fixed):
   $\Xi_k = \arg\min_{\Xi} \ \mathbb{E}\big[d(\Psi_{\phi_{k-1},\Xi}(\mathrm{x}_T),\ \Psi^*(\mathrm{x}_T))\big]$
2. **Optimize coefficients ϕ** (holding Ξ fixed):
   $\phi_k = \arg\min_{\phi} \ \mathbb{E}\big[\min_{\mathrm{x}'_T \in B_r(\mathrm{x}_T)} d(\Psi_{\phi,\Xi_k}(\mathrm{x}'_T),\ \Psi^*(\mathrm{x}_T))\big]$

Time step parametrization: monotonically decreasing via *cumulative softmax*:

$$T = t_0 > t_1 > \cdots > t_N = 0$$

Decoupled evaluation times $t_i^c = t_i + \xi_i^c$ account for score network error.

Typically $K = 8$ alternations suffice; avoids complex joint landscape.

S4S: Key Results

FID (↓) comparison at 5 NFEs

Method	CIFAR-10	FFHQ	ImageNet	MS-COCO
DPM-Solver	15.40	30.29	16.86	22.02
UniPC	16.49	28.16	13.97	18.42
DPM-Solver-v3	17.88	—	—	—
LD3	12.24	14.39	8.36	16.32
S4S	5.64	8.47	5.77	14.17
S4S-Alt	**3.73**	**6.25**	**4.30**	**13.26**

- $\sim$**1.5**$\times$ **improvement** over best training-free methods
- Cost: $<$**1–4 hours** on a single A100 (vs. days for training-based distillation)
- Works with **any pretrained** diffusion model (DDPM, EDM, Stable Diffusion)

S4S: Summary

Key contributions:

- Reframe solver design as *global error minimization* via teacher distillation
- Relaxed objective with theoretical KL guarantee
- Alternating optimization of coefficients + time steps (S4S-Alt)
- 1.5× FID improvement over training-free baselines, <4h on one GPU

Limitations:

- Still requires **multiple NFEs** (4–7); ODE framework retained
- Quality degrades rapidly below 4 NFEs

Natural question

Can we go even further—to a **single step**?

Drifting Models

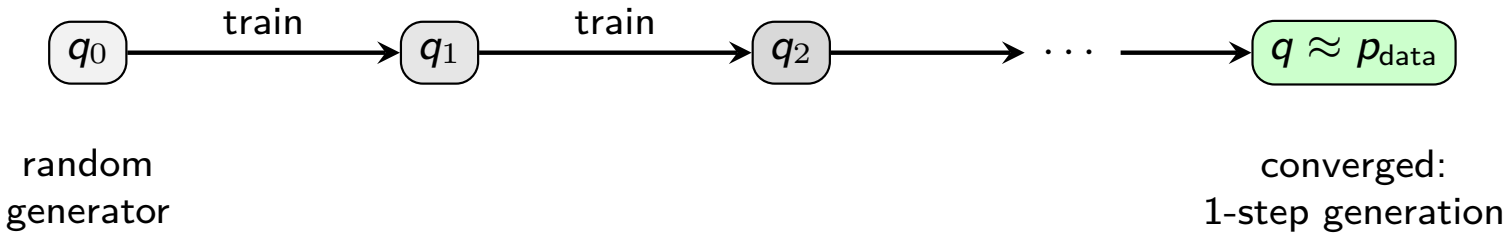
Drifting: Motivation

Diffusion/flow models: build the pushforward iteratively *at inference*.

New idea: evolve the pushforward distribution during *training*.

At inference: a single forward pass

$$\mathsf{x} = f_\theta(\boldsymbol{\epsilon}), \qquad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathsf{0}, \mathsf{I})$$

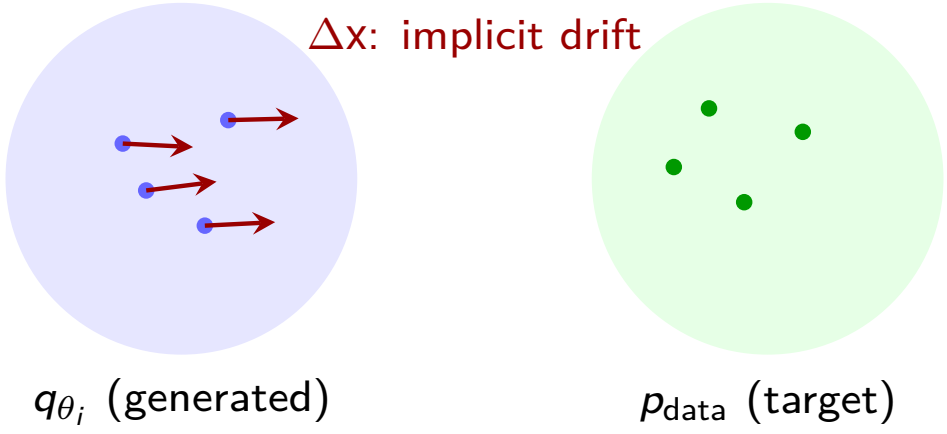


Training-Time Distribution Evolution

During training iteration i, a sample *implicitly drifts*:

$$\mathrm{x}_{i+1} = \mathrm{x}_i + \Delta\mathrm{x}_i, \qquad \Delta\mathrm{x}_i = f_{\theta_{i+1}}(\boldsymbol{\epsilon}) - f_{\theta_i}(\boldsymbol{\epsilon})$$

The network's weight updates **implicitly move samples** in output space.



Key question: can we *design* this movement to match p_{data}?

The Drifting Field

Define a vector field $\mathsf{V}_{p,q} : \mathbb{R}^d \to \mathbb{R}^d$ governing sample movement:

$$\mathsf{x}_{i+1} = \mathsf{x}_i + \mathsf{V}_{p,q}(\mathsf{x}_i)$$

where $p = p_{\text{data}}$ (target), $q = q_\theta$ (current pushforward), $\mathsf{x}_i = f_\theta(\boldsymbol{\epsilon}) \sim q$.

Goal: design V so that repeated application drives $q \to p$.

Requirements for V**:**

1. Points samples *toward* the data distribution
2. Vanishes at equilibrium ($q = p$)
3. Computable from batches of real and generated data

Anti-Symmetry and Equilibrium

Anti-symmetry property:

$$V_{p,q}(x) = -V_{q,p}(x), \qquad \forall\, x$$

Consequence (equilibrium): if $q = p$, then

$$V_{p,p}(x) = -V_{p,p}(x) \;\Rightarrow\; V_{p,p}(x) = 0$$

Interpretation

- Samples **stop moving** when distributions match
- The drifting field is a *self-correcting* signal
- Under generic assumptions: $V \approx 0 \;\Rightarrow\; p \approx q$

Anti-symmetry is **critical**: breaking it (e.g., $1.5V_p^+ - V_q^-$) causes FID to jump from **8.46 to 41**.

Fixed-Point Training Formulation

At equilibrium (optimal $\hat{\theta}$):

$$f_{\hat{\theta}}(\boldsymbol{\epsilon}) = f_{\hat{\theta}}(\boldsymbol{\epsilon}) + \mathsf{V}\big(f_{\hat{\theta}}(\boldsymbol{\epsilon})\big)$$

This motivates the **training loss**:

$$\boxed{\mathcal{L} = \mathbb{E}_{\boldsymbol{\epsilon}}\Big[\, \|f_\theta(\boldsymbol{\epsilon}) - \mathrm{sg}(f_\theta(\boldsymbol{\epsilon}) + \mathsf{V}(f_\theta(\boldsymbol{\epsilon})))\|^2 \,\Big]}$$

- $\mathrm{sg}(\cdot)$: stop-gradient (frozen target, like momentum contrast)
- Loss value $= \mathbb{E}\big[\|\mathsf{V}\|^2\big]$ — directly measures **drift magnitude**
- At convergence: $\mathsf{V} \to \mathsf{0}$, loss $\to 0$
- No back-propagation through V (it depends on q_θ)

Generator "chases" a drifted version of its own output, frozen each iteration.

Kernel-Based Drifting Field

Mean-shift decomposition: attraction + repulsion

Attraction (pull toward data):

$$\mathsf{V}_p^+(\mathsf{x}) = \frac{1}{Z_p}\,\mathbb{E}_{\mathsf{y}^+\sim p}\big[k(\mathsf{x},\mathsf{y}^+)\,(\mathsf{y}^+ - \mathsf{x})\big]$$

Repulsion (push away from other generated samples):

$$\mathsf{V}_q^-(\mathsf{x}) = \frac{1}{Z_q}\,\mathbb{E}_{\mathsf{y}^-\sim q}\big[k(\mathsf{x},\mathsf{y}^-)\,(\mathsf{y}^- - \mathsf{x})\big]$$

Combined field (anti-symmetric by construction):

$$\mathsf{V}_{p,q} = \mathsf{V}_p^+ - \mathsf{V}_q^-$$

Kernel: $k(\mathsf{x},\mathsf{y}) = \exp\big(-\|\mathsf{x}-\mathsf{y}\|/\tau\big)$, normalized via **double softmax**.

Practical Algorithm

Drifting Model Training

Input: Generator f_θ, data p_{data}, kernel temperature τ

For each training iteration:

1. Sample $\boldsymbol{\epsilon} \sim p_\epsilon$
2. $\mathsf{x} \leftarrow f_\theta(\boldsymbol{\epsilon})$ *// generate batch*
3. $\mathsf{y}^+ \sim p_{\text{data}}$ *// positive samples (real data)*
4. $\mathsf{y}^- \leftarrow \mathsf{x}$ *// negative samples (generated)*
5. $\mathsf{V} \leftarrow \textsc{ComputeV}(\mathsf{x}, \mathsf{y}^+, \mathsf{y}^-, \tau)$
6. $\mathcal{L} \leftarrow \|\mathsf{x} - \mathsf{sg}(\mathsf{x} + \mathsf{V})\|^2$
7. Update θ via $\nabla_\theta \mathcal{L}$

$\textsc{ComputeV}$: pairwise distances $\rightarrow$ softmax logits $\rightarrow$ weighted means

Double softmax: row-normalize (over y) and column-normalize (over x), then take $\sqrt{\text{row} \times \text{col}}$.

Feature Space Extension

Kernel on **raw pixels** is too flat for high-dimensional data.

Solution: encode with feature extractor ϕ, compute V in feature space:

$$\mathcal{L} = \sum_j \mathbb{E}\left[\left\|\phi_j(\mathsf{x}) - \mathsf{sg}\big(\phi_j(\mathsf{x}) + \mathsf{V}_j\big)\right\|^2\right]$$

- ϕ_j: multi-scale features (4 scales from ResNet encoder)
- Feature encoding is **training-time only** (not at inference—still 1 NFE!)
- Encoder: ResNet pre-trained via **latent-MAE** (self-supervised)

Encoder quality matters:

Encoder	FID
SimCLR	11.05
MoCo-v2	8.41
Latent-MAE	**3.36**

Classifier-Free Guidance (Training-Time)

Traditional CFG: applied at *inference* in diffusion models.

Drifting CFG: applied at *training* by mixing negative samples.

Negative sample distribution (given class c):

$$\tilde{q}(\cdot|c) = (1-\gamma)\, q_\theta(\cdot|c) + \gamma\, p_{\text{data}}(\cdot|\varnothing)$$

CFG scale: $\alpha = \frac{1}{1-\gamma} \geq 1$

Interpretation: at equilibrium $\tilde{q}(\cdot|c) = p_{\text{data}}(\cdot|c)$, so

$$q_\theta(\cdot|c) = \alpha \cdot p_{\text{data}}(\cdot|c) - (\alpha - 1) \cdot p_{\text{data}}(\cdot|\varnothing)$$

The generator learns a **sharpened** conditional distribution.

Unlike diffusion CFG, the scale α can be varied at test time without retraining.

Drifting: Key Results (ImageNet 256×256)

1-step generation (1 NFE) — comparison with prior art

Method	**Type**	**NFE**	**FID ↓**
Multi-step (for reference):			
SiT-XL/2 + REPA	Flow	250×2	1.42
DiT-XL/2	Diffusion	250	2.27
1-step methods:			
iCT-deep	Consistency	1	3.25
MeanFlow-XL/2	Flow	1	3.43
iMeanFlow-XL/2	Flow	1	1.72
AdvFlow-XL/2	Flow	1	2.38
Drifting B/2	**Drifting**	**1**	**1.75**
Drifting L/2	**Drifting**	**1**	**1.54**

Pixel space: Drifting L/16 achieves FID **1.61** (1 NFE)—pixel-space SOTA.

Drifting: Ablation Studies

Anti-symmetry is critical:

Variant	FID
$V_p^+ - V_q^-$ (default)	**8.46**
$1.5V_p^+ - V_q^-$	41.05
$V_p^+ - 1.5V_q^-$	46.28
V_p^+ only	177.1

Sample allocation:
More positives *and* negatives both help. $N_{\text{pos}} = N_{\text{neg}} = 64$ optimal at batch 4096.

Scaling behavior:

Config	Epochs	FID
B/2, 100ep	100	3.36
B/2, 320ep	320	2.51
B/2, 1280ep	1280	1.75
L/2, 1280ep	1280	**1.54**

Longer training + larger model $\Rightarrow$ consistent improvement. No saturation observed.

Drifting: Beyond Images (Robotics)

Drifting Policy: 1-step replacement for 100-step Diffusion Policy.

Task	Obs.	Diffusion (100)	Drifting (1)
Lift	State	0.98	**1.00**
Can	State	0.96	**0.98**
Can	Visual	0.97	**0.99**
ToolHang	State	0.30	**0.38**
BlockPush (ph. 1)	State	0.36	**0.56**
BlockPush (ph. 2)	State	0.11	**0.16**

- **100**$\times$ **fewer** NFEs with comparable or better performance
- No feature encoder needed—drifting on **raw representations**

Discussion

Open Questions & Future Directions

S4S:

- Extend to SDE solvers? Sample/prompt-specific coefficients?
- Transfer learned coefficients across NFE budgets?

Drifting:

- Can the feature encoder dependency be removed?
- Better convergence guarantees?
- Connections to ideas from optimal transport / variational inference?

Unifying theme:

- The boundary between "training" and "inference" is **blurring**
- Both push toward: *less inference compute, more at design time*